МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу «Операционные системы»

Вариант 10

Управление потоками и их синхронизация: решение системы линейных уравнений методом Гаусса

Выполнил: Якунченков Кирилл Андреевич
Группа: М8О-203БВ-25
Преподаватель: А. С. Ядров

28.10.2026

Москва, 2026

Условие

Цель работы: приобретение практических навыков управления потоками в операционной системе и обеспечения синхронизации между потоками.

Задание: составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, задаётся ключом запуска программы. Количество потоков, используемое программой, демонстрируется стандартными средствами операционной системы. Исследовать зависимость ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Вариант: 10.

1. Решить систему линейных уравнений $Ax = b$ методом Гаусса.
2. Потоки создаются средствами POSIX (`pthread_create`); их число задаётся обязательным ключом `-t`.
3. Система читается со стандартного ввода (с клавиатуры или из файла через перенаправление) либо генерируется случайно (ключ `-g`) для замеров.
4. Число потоков работающей программы показывается через `ps -L`, `/proc/<pid>/status` и `top -H`.

```
строка   элементы                        владелец
  0       a00  a01  a02  ...  a0n | b0   поток 0
  1       a10  a11  a12  ...  a1n | b1   поток 1
  2       a20  a21  a22  ...  a2n | b2   поток 2
  3       a30  a31  a32  ...  a3n | b3   поток 0
  ...                                    поток i mod T

шаг k:  поток 0       выбирает ведущую строку и ставит её на место k
        барьер        ведущая строка готова
        каждый поток вычитает строку k из своих строк с номерами больше k
        барьер        столбец k исключён
```

Распределение строк расширенной матрицы $[A \mid b]$ между потоками ($T = 3$)

Метод решения

Прямой ход метода Гаусса за n шагов приводит матрицу к треугольному виду. На шаге k выбирается ведущая строка с наибольшим $|a_{ik}|$ среди строк $k, \ldots, n-1$ (выбор по столбцу уменьшает ошибки округления) и ставится на место k. Затем из каждой строки ниже вычитается ведущая, умноженная на a_{ik}/a_{kk}. Обратный ход находит $x_{n-1}, \ldots, x_0$. Прямой ход требует около $\frac{2}{3}n^3$ операций, обратный — n^2, поэтому распараллеливается прямой ход.

Строки внутри шага независимы: каждая изменяется только своим потоком и читает лишь ведущую строку. Строка i обрабатывается потоком $i \bmod T$ (циклическое распределение): строки в работе убывают сверху, и при таком распределении у всех потоков до конца остаётся почти одинаковое число строк. Перестановка строк — обмен двух указателей, данные не копируются.

Шаг состоит из двух частей, разделённых барьерами. Поток 0 выбирает и переставляет ведущую строку; барьер; все потоки исключают свои строки; барьер. Без первого барьера потоки читали бы ещё не выбранную ведущую строку, без второго — поток 0 переставлял бы строки, которые другие ещё изменяют.

Результат не зависит от числа потоков: каждый элемент вычисляется одними и теми же операциями в том же порядке. Матрица считается вырожденной, если модуль ведущего элемента не больше $n \cdot \varepsilon \cdot \max |a_{ij}|$, где ε — машинная точность `double`.

Описание программы

Модули

- `common/`: коды возврата, вывод ошибок, разбор числовых ключей.
- `os/`: команда потоков, барьер, монотонные часы, PID/TID, проверка терминала.
- `src/gauss/system.c`: хранение, чтение, генерация системы, невязка.
- `src/gauss/gauss.c`: прямой и обратный ход.
- `src/gauss/main.c`: ключи запуска, замер времени, вывод результата.

Расширенная матрица хранится одним блоком памяти, доступ к строкам — через массив указателей. Система читается со стандартного ввода; если это терминал (`isatty`), программа запрашивает размер и каждую строку. Для замеров ключ `-g n` строит случайную матрицу с $b_i = \sum_j a_{ij}$, точное решение которой — все единицы. Правильность проверяется невязкой $\max_i |(Ax)_i - b_i|$ по копии исходной системы.

Потоки

`os_team_run(T, body, context)` запускает тело в T потоках сразу: индекс 0 выполняет вызывающий (главный) поток, для остальных вызывается `pthread_create`. Поэтому при ключе `-t T` в процессе ровно T потоков. Новые потоки сначала ждут у «стартовых ворот» (mutex и условная переменная), пока не будут созданы все. Если `pthread_create` завершится ошибкой, ворота отменяются: созданные потоки выходят, не входя в барьер, который уже никогда бы не заполнился. Все созданные потоки присоединяются через `pthread_join`.

Синхронизация

Шаги разделяет барьер `pthread_barrier_t`. Поток, пришедший к барьеру, засыпает в ядре (`futex`); последний пришедший будит остальных. Барьер также упорядочивает память: всё, что поток записал до барьера, видно другим после него. Поэтому общие данные (строки, флаг вырожденности) не защищаются блокировками: каждое поле в каждый момент пишет один поток, а читают его другие только после барьера.

Коды возврата

0	штатное завершение
2	неверные ключи запуска
3	ошибка чтения или формата системы
4	ошибка записи решения
5	матрица вырождена: нет единственного решения
6	ошибка создания потока или барьера
7	недостаточно памяти

Результаты

Сборка и проверка

Программа проверена в Ubuntu 22.04 под WSL2 на процессоре AMD Ryzen 5 7535HS (6 ядер, 12 логических процессоров, кэш L3 16 МБ, 8 ГБ памяти, выделенных WSL). Команды выполняются из корня репозитория:

```
make
make test
```

В журнале `logs/tests.txt`: 12 сценариев, 46 успешных проверок. Проверяются решение с перестановкой строк, совпадение ответа при разном числе потоков, вырожденные и несовместные системы, ошибки формата, ключей и записи (`/dev/full`), TID потоков, число потоков в `/proc`, отказ `pthread_create` при `ulimit -u 1` и ручной ввод с терминала (через псевдотерминал `script`).

```
  ok   8.5 7 threads: the same residual
  ok   8.6 16 threads: the same residual
  ok   9.1 5 threads report themselves
  ok   9.2 the 5 TIDs are different
  ok   9.3 thread 0 is the main thread
  ok   10.1 -t 1: at most 1 threads in /proc, code 0
  ok   10.2 -t 4: at most 4 threads in /proc, code 0
  ok   10.3 -t 6: at most 6 threads in /proc, code 0
  ok   11.1 pthread_create fails: code 6
  ok   11.2 the reason is reported
  ok   12.1 exit code 0 without EOF after the last row
  ok   12.2 the size and both rows are asked for
  ok   12.3 the solution 2 1 is printed

TOTAL: passed 46, failed 0
```

Окончание журнала тестов

Демонстрационный сеанс

```
$ ./gauss -t 3 < system4.txt
gauss: pid=1011 tid=1011 threads=3
gauss: thread 0/3 tid=1011
gauss: thread 2/3 tid=1013
gauss: thread 1/3 tid=1012
solved: n=4 threads=3 time=0.646 ms residual=1.776e-15
1.666666667
2.060606061
```

```
2.575757576
3.696969697
gauss exit code: 0
$ ./gauss -t 6 -g 2000
gauss: pid=1014 tid=1014 threads=6
gauss: thread 0/6 tid=1014
gauss: thread 4/6 tid=1018
gauss: thread 2/6 tid=1016
gauss: thread 5/6 tid=1019
gauss: thread 3/6 tid=1017
gauss: thread 1/6 tid=1015
solved: n=2000 threads=6 time=1075.563 ms residual=9.813e-12
$ printf '2
1 1 1
1 1 2
' | ./gauss -t 2
gauss: pid=1021 tid=1021 threads=2
gauss: thread 0/2 tid=1021
gauss: thread 1/2 tid=1022
the matrix is singular: the system has no unique solution
gauss exit code: 5
```

Сеанс: пример 4×4, случайная система 2000×2000, вырожденная система

Строки, начинающиеся с `$`, добавлены при оформлении: это команды запусков из `logs/demo.txt`. Решение примера — $x = (55, 68, 85, 122)/33$. Поток 0 имеет TID, равный PID: главный поток участвует в вычислениях.

Число потоков средствами ОС

```
$ ./gauss -t 4 -g 3000 &
gauss: pid=951 tid=951 threads=4
gauss: thread 2/4 tid=996
gauss: thread 0/4 tid=951
gauss: thread 1/4 tid=995
gauss: thread 3/4 tid=997

$ ps -L -o pid,lwp,nlwp,psr,stat,comm -p 951
    PID      LWP  NLWP PSR STAT COMMAND
    951      951     4   6 Rl+  gauss
    951      995     4   2 Rl+  gauss
    951      996     4   8 Rl+  gauss
    951      997     4   5 Rl+  gauss

$ grep Threads /proc/951/status
Threads:  4

$ ls /proc/951/task
951 995 996 997

$ top -H -b -n 1 -p 951 | tail -n +7
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
    951 kirill    20   0  168968 142464   1792 R  99.9   1.8   0:00.91 gauss
    995 kirill    20   0  168968 142464   1792 R  99.9   1.8   0:00.68 gauss
    996 kirill    20   0  168968 142464   1792 R  93.3   1.8   0:00.69 gauss
    997 kirill    20   0  168968 142464   1792 R  93.3   1.8   0:00.68 gauss
solved: n=3000 threads=4 time=3658.681 ms residual=3.032e-11

exit code: 0
```

Потоки работающей программы при `-t 4` (`tests/threads_demo.sh`)

`ps -L` выводит по строке на поток: столбец LWP — TID, NLWP — число потоков процесса, PSR — номер процессора, на котором поток выполнялся последним. Все четыре потока работают на разных процессорах. Поле `Threads` в `/proc/<pid>/status` и каталог `/proc/<pid>/task` показывают те же 4 потока, `top -H` — загрузку каждого. Тест 10 опрашивает `/proc/<pid>/status` каждые 10 мс: при `-t 1`, `-t 4`, `-t 6` наибольшее число потоков равно значению ключа.

Системные вызовы

```
1  1026  clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|
       CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0
       x716a9c1ff910, parent_tid=0x716a9c1ff910, exit_signal=0, stack=0x716a9b9ff000,
       stack_size=0x7fff00, tls=0x716a9c1ff640} => {parent_tid=[1027]}, 88) = 1027
2  1027  futex(0x7ffff57493d0, FUTEX_WAIT_BITSET_PRIVATE|FUTEX_CLOCK_REALTIME, 0, NULL,
       FUTEX_BITSET_MATCH_ANY <unfinished ...>
3  1026  clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|
       CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0
       x716a9b9fe910, parent_tid=0x716a9b9fe910, exit_signal=0, stack=0x716a9b1fe000,
       stack_size=0x7fff00, tls=0x716a9b9fe640} => {parent_tid=[1028]}, 88) = 1028
4  1026  futex(0x7ffff57493d0, FUTEX_WAKE_PRIVATE, 2147483647) = 1
5  1027  <... futex resumed>)                = 0
6  1027  futex(0x7ffff5749380, FUTEX_WAKE_PRIVATE, 1) = 0
7  1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 0, NULL <unfinished ...>
8  1027  futex(0x716a9c41ca60, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
9  1028  futex(0x716a9c41ca60, FUTEX_WAKE_PRIVATE, 1) = 1
10 1027  <... futex resumed>)                = 0
11 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 0, NULL <unfinished ...>
12 1027  futex(0x716a9c41ca60, FUTEX_WAKE_PRIVATE, 1) = 0
13 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
14 1026  <... futex resumed>)                = 0
15 1028  <... futex resumed>)                = 0
16 1027  <... futex resumed>)                = 2
17 1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 3, NULL <unfinished ...>
18 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 3, NULL <unfinished ...>
19 1026  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
20 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
21 1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 6, NULL <unfinished ...>
22 1028  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
23 1027  <... futex resumed>)                = 0
24 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 6, NULL <unfinished ...>
25 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
26 1028  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
27 1026  <... futex resumed>)                = 0
28 1027  <... futex resumed>)                = 1
29 1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 9, NULL <unfinished ...>
30 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 9, NULL <unfinished ...>
31 1026  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
32 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
33 1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 12, NULL <unfinished ...>
34 1028  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
35 1027  <... futex resumed>)                = 0
36 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 12, NULL <unfinished ...>
37 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
38 1028  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
39 1026  <... futex resumed>)                = 0
40 1027  <... futex resumed>)                = 1
41 1026  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 15, NULL <unfinished ...>
42 1028  futex(0x5e51ec54c3e4, FUTEX_WAIT_PRIVATE, 15, NULL <unfinished ...>
43 1026  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
44 1027  futex(0x5e51ec54c3e4, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>
45 1026  futex(0x716a9c1ff910, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 1027, NULL,
       FUTEX_BITSET_MATCH_ANY <unfinished ...>
46 1028  <... futex resumed>)                = -1 EAGAIN (Resource temporarily unavailable)
47 1027  <... futex resumed>)                = 0
```

```
1028  exit(0)                           = ?
1027  exit(0 <unfinished ...>
1028  +++ exited with 0 +++
1027  <... exit resumed>)               = ?
1026  <... futex resumed>)              = 0
1027  +++ exited with 0 +++
1026  +++ exited with 0 +++
```

Трассировка `strace -f -e trace=clone3,futex,exit ./gauss -t 3 -g 3`

Главный поток (1026) дважды вызывает `clone3` с флагами `CLONE_VM|CLONE_THREAD`: новые задачи разделяют адресное пространство и принадлежат тому же процессу. Первые `futex` — стартовые ворота (условная переменная). Далее на каждом барьере потоки засыпают (`FUTEX_WAIT`) на одном адресе, а последний пришедший будит всех (`FUTEX_WAKE` с числом 2147483647). Ответ `EAGAIN` означает, что барьер открылся раньше, чем поток успел заснуть. Система 3×3 даёт 3 шага по 2 барьера. Завершение потоков — `exit`; главный поток ждёт их в `pthread_join` (`futex` на адресе TID).

Исследование ускорения и эффективности

Ускорение $S_p = T_1/T_p$, эффективность $E_p = S_p/p$, где T_p — время решения в p потоках. T_1 измерено той же программой с `-t 1`. Каждая точка — медиана пяти запусков на одной случайной системе (`tests/bench.sh`, журнал `logs/bench.txt`). Измеряется только решение: генерация и проверка в замер не входят.

n	T_1, мс	ускорение S_p при p потоках						
		2	3	4	6	8	12	16
100	0,21	0,04	0,03	0,02	0,01	0,01	0,01	0,005
200	0,99	0,09	0,06	0,04	0,03	0,02	0,02	0,01
500	12,4	0,36	0,27	0,21	0,14	0,11	0,08	0,06
1000	96,2	0,90	0,79	0,67	0,51	0,39	0,24	0,21
2000	1705	1,53	1,84	**1,91**	1,77	1,65	1,49	1,25
3000	7151	1,42	1,91	**2,24**	1,79	1,95	1,83	1,84

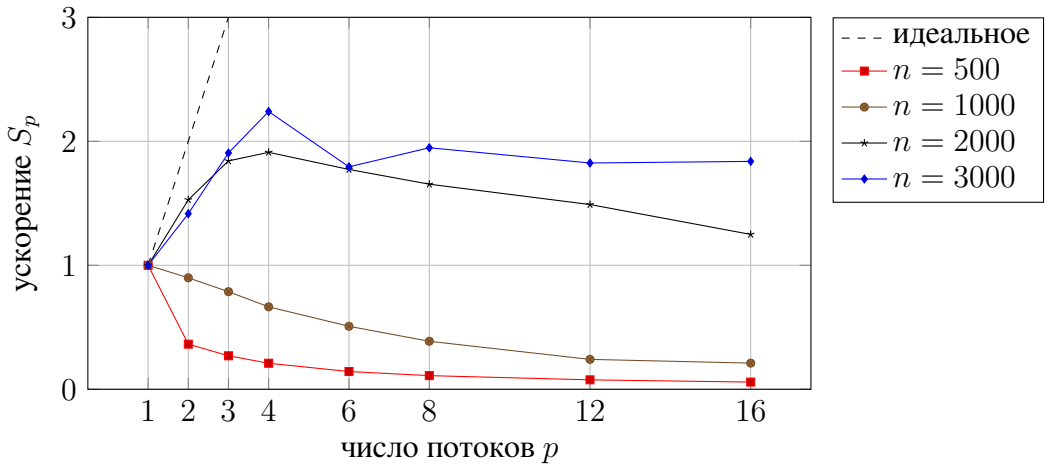


Рис. 1: Ускорение в зависимости от числа потоков и размера системы

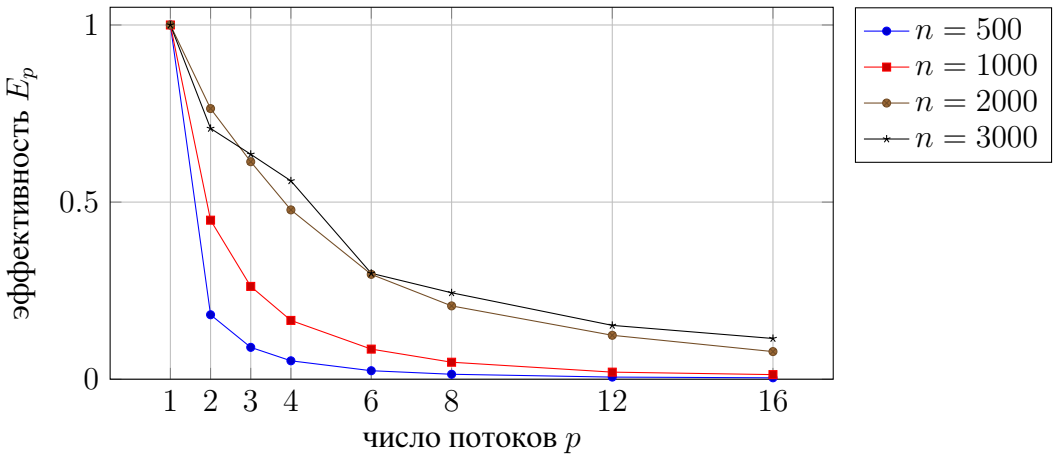


Рис. 2: Эффективность в зависимости от числа потоков и размера системы

Объяснение результатов

Цена синхронизации. На каждом шаге два барьера, всего $2n$. У `pthread_barrier_t` ожидающий поток засыпает в ядре, а последний пришедший будит остальных; каждое пробуждение — системный вызов и планирование потока, в WSL это десятки микросекунд. При $n = 100$ решение в одном потоке занимает 0,21 мс, а в двух — 5,5 мс: 200 барьеров стоят около 5 мс. С ростом p надо будить больше потоков, и время растёт примерно линейно (при $n = 100$: 5,5; 11,6; 22,5 мс для 2, 4, 8 потоков).

Малые и средние системы ($n \leq 1000$). Работа шага равна примерно $(n - k)^2$ операций и делится на p, а затраты на барьеры от n почти не зависят. Пока шаг короче пробуждения потоков, параллельная версия медленнее последовательной; при $n = 1000$ два потока уже почти сравнялись с одним ($S = 0{,}90$).

Большие системы ($n = 2000$–3000). Работа шага больше затрат на синхронизацию, и потоки дают выигрыш: до 1,9 при $n = 2000$ и 2,2 при $n = 3000$ на 4 потоках. Дальше рост ограничивают две причины. Матрица (32 и 72 МБ) не помещается в кэш L3, и каждый шаг заново читает и пишет оставшуюся часть матрицы из оперативной памяти; пропускная способность памяти общая для всех ядер. Это видно уже в одном потоке: при росте n с 1000 до 2000 работа растёт в 8 раз, а время — в 18. Кроме того, последние шаги почти не содержат работы, а барьеры остаются.

Число потоков больше 4–6. У процессора 6 физических ядер; два логических процессора одного ядра делят его вычислительные блоки и кэши. При $p > 6$ вычислений не прибавляется, а барьеров на поток больше; при $p = 16$ потоков больше, чем процессоров, и потоки ждут ещё и очереди на процессор. Эффективность монотонно падает с ростом p при любом n.

Последовательная часть. Обратный ход, поиск масштаба матрицы и выбор ведущей строки ($O(n^2)$ в сумме) выполняются одним потоком, но при $n = 3000$ занимают миллисекунды из секунд. Значения элементов матрицы на время почти не влияют: метод выполняет одинаковое число операций для любой невырожденной плотной матрицы.

Разброс отдельных замеров в виртуальной машине WSL достигает десятков процентов (например, $n = 3000$, $p = 6$ выбивается из ряда).

Выводы

Прямой ход метода Гаусса распараллелен на потоках POSIX: число одновременно работающих потоков задаётся ключом `-t` и подтверждается средствами ОС (`ps -L`, `/proc/<pid>/status`, `top -H`, `strace`). Строки распределены между потоками циклически, шаги разделены барьером `pthread_barrier_t`, общие данные защищены порядком «запись до барьера — чтение после». Ответ не зависит от числа потоков. В журнале автотестов успешно выполнены 46 проверок, включая отказ `pthread_create` без зависания.

Многопоточность окупается только на больших системах: ускорение 1,9 при $n = 2000$ и 2,2 при $n = 3000$ на 4 потоках. Для $n \leq 1000$ засыпание и пробуждение потоков на $2n$ барьерах дороже полезной работы шага, и программа в нескольких потоках работает медленнее. Сверху ускорение ограничивают пропускная способность памяти и число физических ядер.

Ограничения: обратный ход последователен; замеры выполнены в виртуальной машине WSL и имеют заметный разброс.

Значимые фрагменты программы

Запуск команды потоков: главный поток работает как поток 0, остальные создаются `pthread_create` и ждут у стартовых ворот; все присоединяются `pthread_join`:

```
    /* index 0 is this thread so only count-1 new ones */
    int created = 0;
    for (int index = 1; index < count; ++index) {
        members[index].gate  = &gate;
        members[index].index = index;
        error = pthread_create(&threads[index], NULL, team_thread, &members[index]);
        if (error != 0) {
            break;
        }
        ++created;
    }

    gate_set(&gate, (error == 0) ? GATE_OPEN : GATE_CANCELLED);
    if (error == 0) {
        body(0, context);
    }

    for (int index = 1; index <= created; ++index) {
        const int joined = pthread_join(threads[index], NULL);
        if (joined != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(joined));
        }
    }
```

Стартовые ворота: ожидание на условной переменной и открытие:

```
static gate_state gate_wait(team_gate* gate)
{
    pthread_mutex_lock(&gate->mutex);
    /* loop because of spurious wakeups */
    while (gate->state == GATE_CLOSED) {
        pthread_cond_wait(&gate->changed, &gate->mutex);
    }
    const gate_state state = gate->state;
    pthread_mutex_unlock(&gate->mutex);
    return state;
}

static void gate_set(team_gate* gate, gate_state state)
{
```

```
    pthread_mutex_lock(&gate->mutex);
    gate->state = state;
    pthread_cond_broadcast(&gate->changed);
    pthread_mutex_unlock(&gate->mutex);
}
```

Шаг прямого хода в каждом потоке: выбор ведущей строки, два барьера:

```
static void gauss_thread(int index, void* context)
{
    gauss_task* task = context;

    fprintf(stderr, "gauss: thread %d/%d tid=%lld\n", index, task->threads, os_thread_id
        ());

    for (int k = 0; k < task->system->size; ++k) {
        if (index == 0 && choose_pivot(task, k) < 0) {
            task->singular = 1;
        }
        os_barrier_wait(task->barrier);       /* pivot row is ready */
        if (task->singular) {
            return;
        }
        eliminate_rows(task, k, index);
        os_barrier_wait(task->barrier);       /* column k is done */
    }
}
```

Исключение в строках потока (циклическое распределение):

```
/* row i belongs to thread i % threads so the work stays even until the end */
static void eliminate_rows(gauss_task* task, int k, int index)
{
    const int     n     = task->system->size;
    double**      rows  = task->system->rows;
    const double* pivot = rows[k];

    for (int i = k + 1; i < n; ++i) {
        if (i % task->threads != index) {
            continue;
        }
        double*       row    = rows[i];
        const double factor = row[k] / pivot[k];
        for (int j = k + 1; j <= n; ++j) {
            row[j] -= factor * pivot[j];
        }
        row[k] = 0.0;
    }
}
```

Полные исходники находятся в `common/`, `os/`, `src/`. Копии в `docs/inc/` используются только для сборки отчёта.

Литература

1. Man-страницы Linux: pthread_create(3), pthread_join(3), pthread_barrier_wait(3p), pthread_cond_wait(3p), futex(2), clone(2), proc(5), ps(1).

2. Голуб Дж., Ван Лоун Ч. Матричные вычисления. — М.: Мир, 1999. Гл. 3.

3. Репозиторий курса: Operation-Systems.

4. Шаблон отчёта: OS_report_template.